

FORMAN CHRISTIAN COLLEGE

A Chartered University

Embedded Systems CSCS306

Class Project

This class project can be completed in groups. Group may have a maximum of three members.

Make sure you submit the working hardware and a well formatted report on the day specified. Please do not add code in your report. Just make sure that you clearly write the logic implemented in your code.

In case if your hardware is not working, no partial marks will be granted.

This project contributes 15% graded marks towards the overall grade.

Code, Implementation and viva

90%

Report with your code explanation

10%

Group Members:

Submission Dead Line: Monday, January 07, 2025 at 11:00 am in class. You must bring your working hardware and report of the project with you.

Project Tasks

In this project you will extend the functionality of lab 10.

Lab 10 was about uploading the status of an LED on google sheet. Repeat the procedure here and on top of it (whether using a mobile app or web app) download the uploaded data such that user should provide a time span and your system should display the total time the LED remained on during that time.

Following are some pointers taken from the Internet (not properly tested) which you may find helpful. You are not bound to use the same. Come up with your own logic.

It is highly recommended to use a mobile app (or a web browser) where user can provide inputs in a way that is different from the scheme discussed below. Similarly, the total ON time should also be displayed on the same platform (browser or mobile) with appropriate formatting.

Keep in mind that your code should be portable to any electric home fixture (for example set of LEDs in a room, fans, air conditioners etc) so that we can get the ON time of these fixtures and can compute the total power consumed in a given period of time.

Task 1: Marks [25]

Following is a set of code (google script and ESP32 code, both are not tested fully) provided for you to get a jump start. Type/copy the code and if there are some erroneous lines, make correction and run it. Make sure you develop a good understanding of the code provided.

To download the "ON time" of an LED based on a specified date and time range from a Google Sheet, you can create a Google Apps Script web app that accepts the date and time range as parameters, filters the records from the Google Sheet, and returns the total ON time within the specified range.

Steps to Accomplish This:

1. Create the Google Sheet that stores the LED status along with timestamps.
2. Write a Google Apps Script that accepts date and time range as parameters, filters the data, and calculates the total ON time for the LED.
3. Make an HTTP request from your ESP32 (or another platform) to the Google Apps Script web app, passing the date and time range, and receiving the calculated-ON time in response.

Step 1: Structure of Google Sheet

Assume your Google Sheet has the following structure:

Date	Time	Status
2024-11-01	08:00:00	ON
2024-11-01	09:00:00	OFF
2024-11-01	10:00:00	ON
2024-11-01	11:00:00	OFF
...	...	...

Where:

- Date is the date of the status update.
- Time is the time when the status was updated.
- Status indicates whether the LED was "ON" or "OFF" at that time.

Step 2: Write Google Apps Script to Calculate ON Time

This Google Apps Script will calculate the total ON time based on a given date and time range. The script will process the data from the Google Sheet and return the total time the LED was "ON" within that range.

1. Open the Google Sheet.
2. Go to Extensions > Apps Script.
3. Replace the default script with the following code:

```
function doGet(e) {
  var startDate = e.parameter.startDate; // Start date (YYYY-MM-DD)
  var startTime = e.parameter.startTime; // Start time (HH:MM:SS)
  var endDate = e.parameter.endDate;    // End date (YYYY-MM-DD)
  var endTime = e.parameter.endTime;    // End time (HH:MM:SS)

  // Combine date and time into full datetime string (e.g., "2024-11-01
  08:00:00")
  var startDateTime = new Date(startDate + " " + startTime);
  var endDateTime = new Date(endDate + " " + endTime);

  var sheet = SpreadsheetApp.getActiveSpreadsheet().getActiveSheet();
  var data = sheet.getDataRange().getValues(); // Get all rows of the
  sheet

  var totalOnTimeMillis = 0; // Variable to accumulate the total ON
  time in milliseconds

  // Loop through each row in the sheet
  for (var i = 1; i < data.length; i++) {
    var row = data[i];
    var recordDate = new Date(row[0] + " " + row[1]); // Combine date
    and time
    var status = row[2]; // LED status ("ON" or "OFF")

    // Check if the record falls within the specified date and time range
    if (recordDate >= startDateTime && recordDate <= endDateTime &&
    status === "ON") {
      // If LED is ON, count the time until the next OFF status or the
      end time
    }
  }
}
```

```

        var nextRecordDate = (i + 1 < data.length) ? new Date(data[i + 1][0] + " " + data[i + 1][1]) : endDateTime;

        // Calculate ON time duration in milliseconds
        var onDurationMillis = nextRecordDate - recordDate;
        totalOnTimeMillis += onDurationMillis; // Add to total ON time
    }
}

// Convert total ON time from milliseconds to hours, minutes, and seconds
var totalOnTimeSeconds = totalOnTimeMillis / 1000;
var hours = Math.floor(totalOnTimeSeconds / 3600);
var minutes = Math.floor((totalOnTimeSeconds % 3600) / 60);
var s3seconds = Math.floor(totalOnTimeSeconds % 60);

// Return the result as a JSON response
var result = {
    "hours": hours,
    "minutes": minutes,
    "seconds": seconds
};

return ContentService.createTextOutput(JSON.stringify(result))
    .setMimeType(ContentService.MimeType.JSON);
}

```

Step 3: Deploy the Script as a Web App

1. After writing the script, click on Deploy > Test deployments > Select type > Web app.
2. Set Execute as to Me and Who has access to Anyone.
3. Click Deploy and authorize the app if prompted.
4. After deploying, you'll get a Web App URL that you can use in your ESP32 code to make HTTP requests.

Step 4: ESP32 Code to Fetch the Total ON Time

Now, on the ESP32, you can send HTTP GET requests to this Google Apps Script Web App, passing the date and time range parameters (e.g., startDate, startTime, endDate, endTime), and receive the total ON time.

1. Install Required Libraries:
 - o WiFi.h for Wi-Fi connection.
 - o HTTPClient.h for HTTP requests.
 - o ArduinoJson.h to parse the JSON response.

2. ESP32 Code:

```

#include <WiFi.h>
#include <HTTPClient.h>
#include <ArduinoJson.h>

// Replace with your Wi-Fi credentials
const char* ssid = "YOUR_SSID";
const char* password = "YOUR_PASSWORD";

// Replace with your Google Apps Script web app URL

```

```

const char* googleScriptURL = "YOUR_GOOGLE_SCRIPT_URL";

void setup() {
    Serial.begin(115200);

    // Connect to Wi-Fi
    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(1000);
        Serial.println("Connecting to WiFi...");
    }
    Serial.println("Connected to WiFi");
}

void loop() {
    // Define the date and time range (example: 2024-11-01 08:00:00 to
    2024-11-01 10:00:00)
    String startDate = "2024-11-01";
    String startTime = "08:00:00";
    String endDate = "2024-11-01";
    String endTime = "10:00:00";

    // Construct the URL with parameters for start date, start time, end
    date, and end time
    String url = googleScriptURL + "?startDate=" + startDate +
    "&startTime=" + startTime + "&endDate=" + endDate + "&endTime=" +
    endTime;

    HTTPClient http;
    http.begin(url); // Initialize the HTTP request
    int httpStatusCode = http.GET(); // Send GET request

    if (httpStatusCode > 0) {
        String response = http.getString(); // Get the response as a string
        Serial.println(response); // Print the raw response

        // Parse the JSON response
        DynamicJsonDocument doc(1024);
        deserializeJson(doc, response);

        int hours = doc["hours"];
        int minutes = doc["minutes"];
        int seconds = doc["seconds"];

        // Print the total ON time
        Serial.print("Total ON time: ");
        Serial.print(hours);
        Serial.print(" hours ");
        Serial.print(minutes);
        Serial.print(" minutes ");
        Serial.print(seconds);
        Serial.println(" seconds.");
    } else {

```

```
    Serial.println("Error: Unable to fetch data from Google Sheets.");  
  }  
  http.end(); // Close HTTP connection  
  
  delay(60000); // Wait 1 minute before requesting again  
}
```

Task 2: Marks [35]

Repeat the above task. This time, instead of displaying the ON time on serial monitor, you should use a mobile app to display the total ON time in an appropriate format. Use App Inventor or any other similar tool to create the mobile app. We have already used one in our labs.

Your application should first ask user to provide a range of time. Given the input, application should display the ON time of the LED from data uploaded on google sheet.

Task 3: Marks [30]

Repeat Task 2 and this time connect four LEDs to your esp32 and upload the ON time of each on google sheet. Redo the mobile app so that you can download the ON time of any LED (given the LED number and time range) on your mobile application.